

Reverse engineering geometrical reasoning of VLMs and toy models

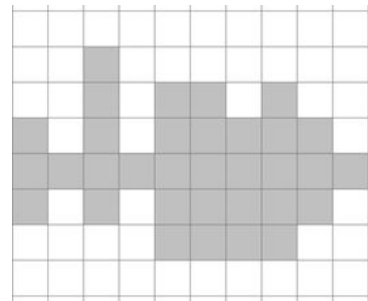
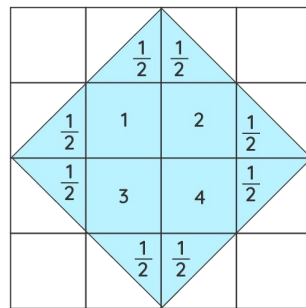
Team Name: see_en_deez

Team Members: Anika, Radhikesh, Ujjwal

Project Tags: Mechanistic Interpretability, VLMs, Toy models, Circuit Recognition, Reverse Engineering

Recap

Problem Statement



The fundamental idea:

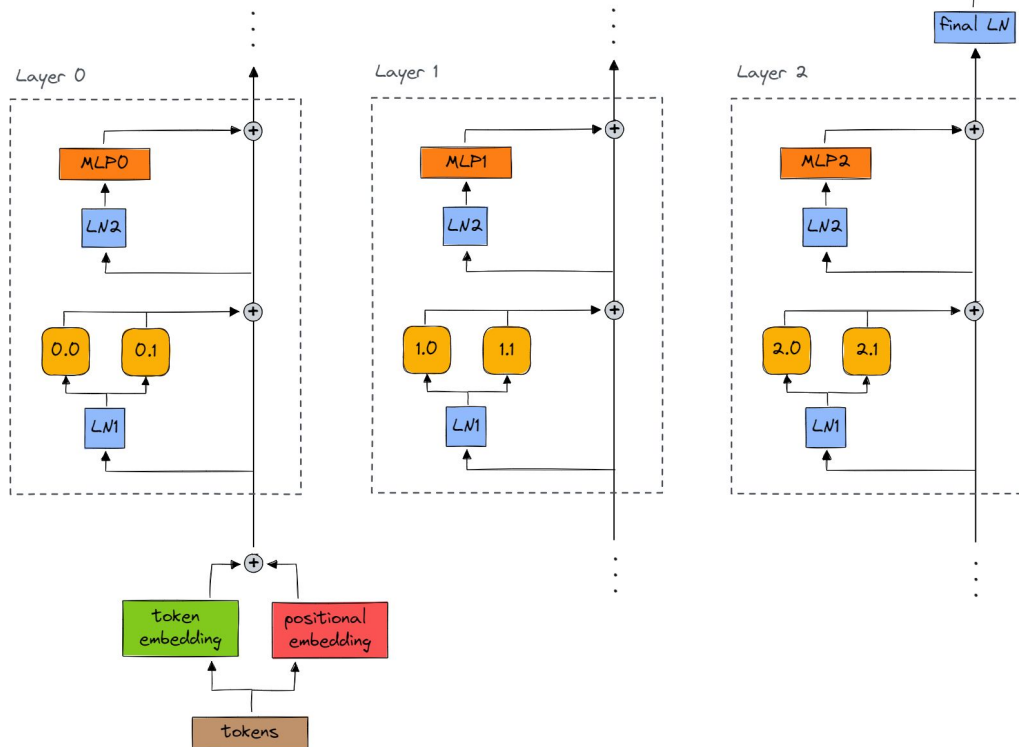
Do toy models, trained on well-understood algorithmic tasks, reliably rediscover known algorithms for solving them?

Our idea:

We want to try a different flavour of algorithms—Geometric algorithms like area and perimeter calculations. There are multiple ways a model could execute these and we want to find out which ones it uses and when (model and input style).

Toy Transformer ✓

Our model:



```
cfg = transformer_lens.HookedTransformerConfig(  
    n_ctx=len(df["word_art"][0]) + 2,  
    model=56,  
    d_head=28,  
    n_heads=2,  
    d_mlp=56,  
    n_layers=3,  
    attention_dir="bidirectional",  
    act_fn="relu",  
    d_vocab=len(VOCAB) + 3,  
    d_vocab_out=df["perimeter"].max() + 1,  
    use_attn_result=True,  
    use_hook_tokens=True,  
    device=device,  
)
```

We are using the [TransformerLens](#) library.

Experiments and Conclusions - Perimeter task

- Our basic 3 layer, 2 heads per layer toy transformer achieves **>99% test accuracy** on the perimeter task. *[Already Done]*
-Uhm, expected, it only has to count the 1's in the sentence given to it. *It's not that lame though.*
- The transformer here predicts the class from [0, max_perimeter]; getting the exact count with such high accuracy is a decent result tbh.
- However this task sets a baseline for later tasks (segment length, area etc).
- We can now start applying techniques from the arena notebooks to interpret these results.

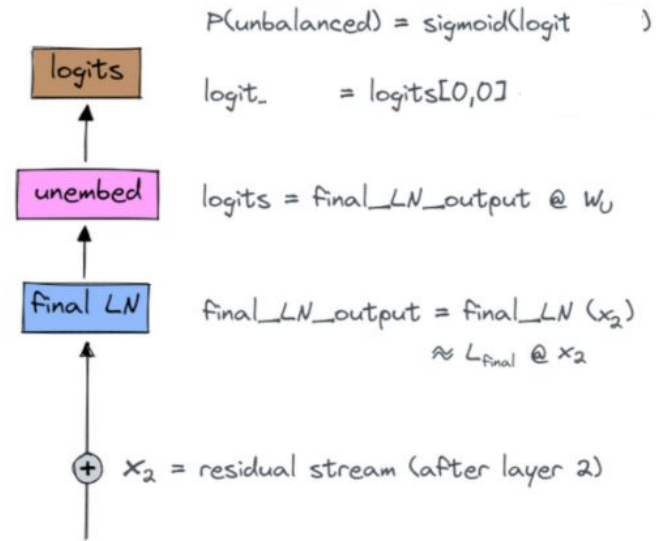
```
Moving model to device: cuda
Epoch 1/25, Loss: 1.7677, LR: 0.000100
Epoch 2/25, Loss: 0.4255, LR: 0.000100
Epoch 3/25, Loss: 0.3592, LR: 0.000100
Epoch 4/25, Loss: 0.7683, LR: 0.000100
Epoch 5/25, Loss: 0.1446, LR: 0.000100
Epoch 6/25, Loss: 0.1159, LR: 0.000100
Epoch 7/25, Loss: 1.6058, LR: 0.000100
Epoch 8/25, Loss: 0.9295, LR: 0.000100
Epoch 9/25, Loss: 0.3922, LR: 0.000100
Epoch 10/25, Loss: 0.2373, LR: 0.000100
Epoch 11/25, Loss: 0.1109, LR: 0.000100
Epoch 12/25, Loss: 0.1805, LR: 0.000100
Epoch 13/25, Loss: 0.1268, LR: 0.000100
Epoch 14/25, Loss: 0.8355, LR: 0.000100
Epoch 15/25, Loss: 0.3544, LR: 0.000100
Epoch 16/25, Loss: 0.0724, LR: 0.000100
Epoch 17/25, Loss: 0.0475, LR: 0.000010
Epoch 18/25, Loss: 0.0446, LR: 0.000010
Epoch 19/25, Loss: 0.0415, LR: 0.000010
Epoch 20/25, Loss: 0.0385, LR: 0.000010
Epoch 21/25, Loss: 0.0358, LR: 0.000010
Epoch 22/25, Loss: 0.0333, LR: 0.000010
Epoch 23/25, Loss: 0.0314, LR: 0.000001
Epoch 24/25, Loss: 0.0312, LR: 0.000001
Epoch 25/25, Loss: 0.0309, LR: 0.000001
```

Accuracy: 0.9932

Updates

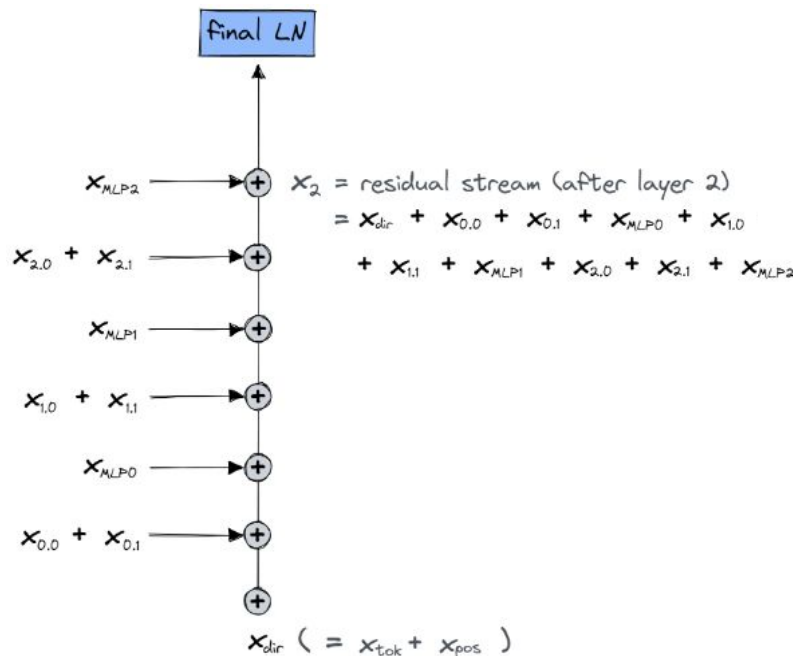
What even is important? - Component Attribution

- Sample a few things from the dataset randomly. We will do a forward pass through the model and see how logits for each class light up.
- We will then assume that the logits were obtained through a series of **linear transformations**. $\text{logits} \leftarrow \text{LN}(x_2 @ W_U)$.
- Only issue : **LayerNorm is non-linear**. Lite, we do a linear fit ($r^2 > 0.94$). And then we keep seeing the dot product of individual components' output with the unembedding matrix.



Per Component Attribution

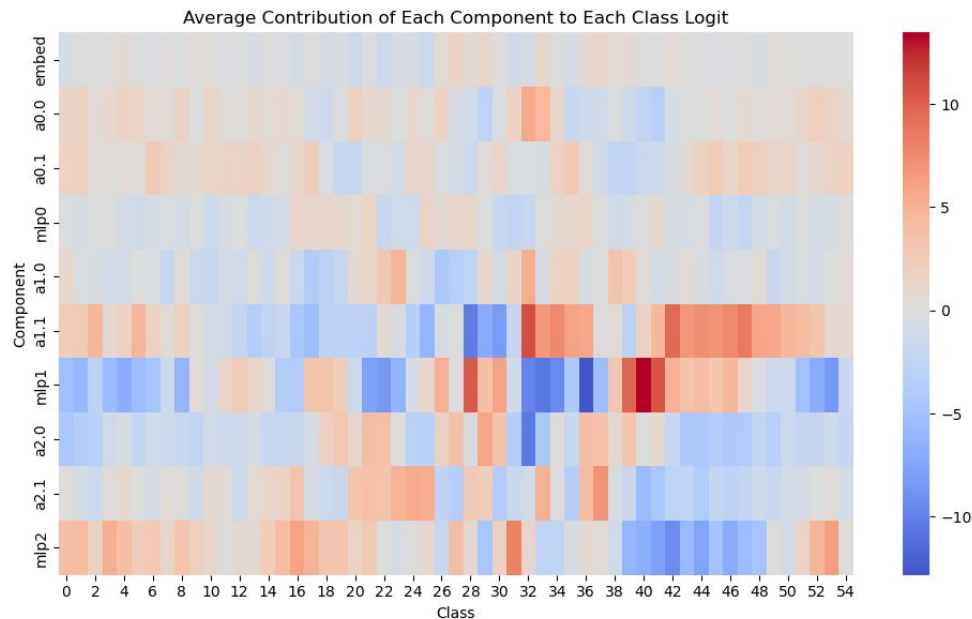
- How much does the output of component c (at position 0 in the sequence for example b) push the residual stream in the direction of class k 's logit vector? -> **Residual (x2)**
- This is simply the dot product of x_2 with the transposed unembed matrix ($W_U.T$ is a linear transformation from $d_model \rightarrow num_classes$).



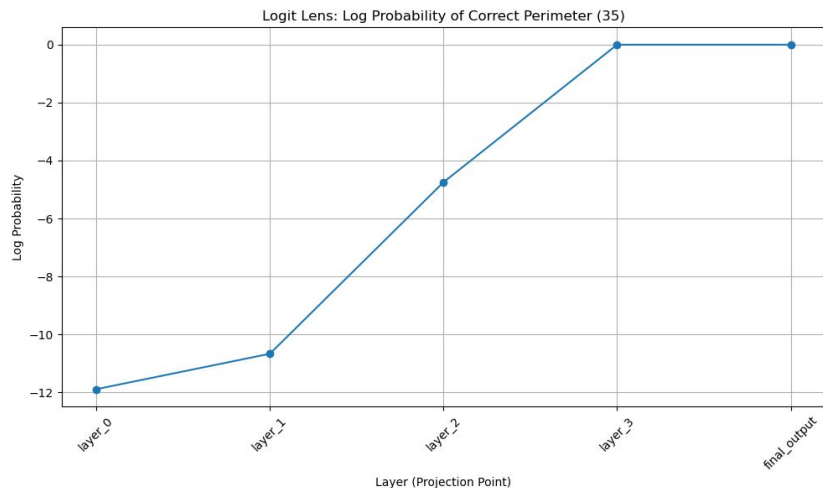
Perimeter Transformer

Component Attribution - Perimeter

- Thus, for each model component and for each class (perimeter value in the sampled data); we see how much the component *pushes* it in the direction.
- Observations:
 - A1.1, A2.0, A2.1 and MLP1 chad lighting up crazy?
 - Early layers useless?



NNSight - Layerwise perf



layer_0: Log prob of correct perimeter (35) = -11.8862

layer_1: Log prob of correct perimeter (35) = -10.6642

layer_2: Log prob of correct perimeter (35) = -4.7482

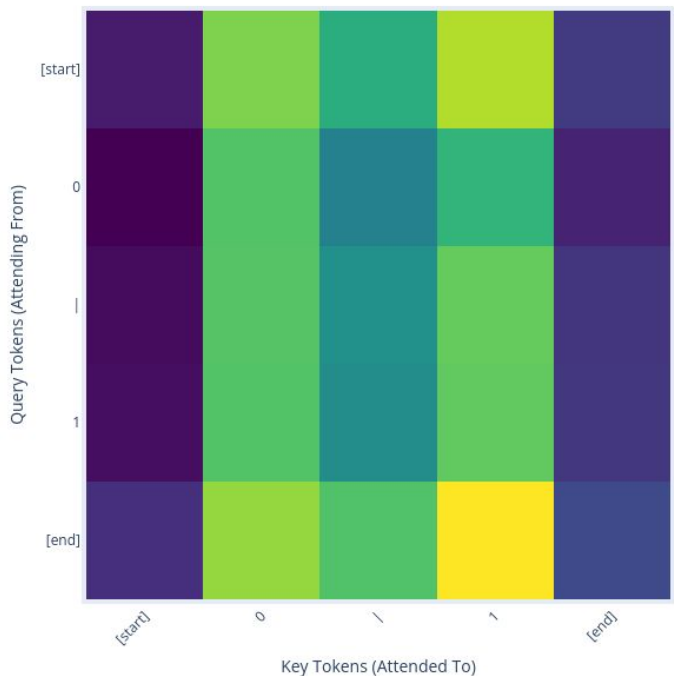
layer_3: Log prob of correct perimeter (35) = -0.0046

final_output: Log prob of correct perimeter (35) = -0.0045

- Model learns refinement over layers.
- Higher probs in earlier layers could show that it learns some sort of heuristics initially *[but it doesn't do that here]*
- We need to inspect the model more closely; *what tokens are being attended to* by each component of our model?

QK interp? 🤔

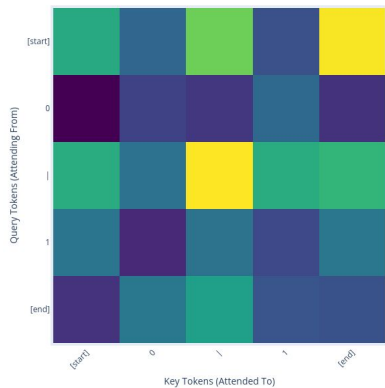
Layer 2, Head 1 Attention Pattern



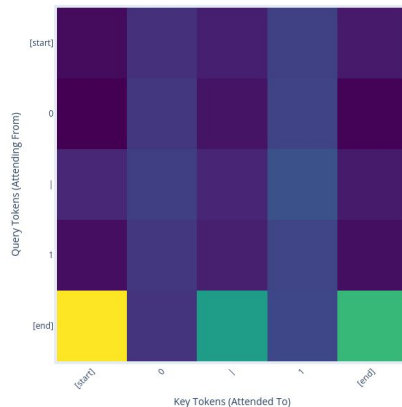
- Traces the model and extracts attention patterns from each layer.
- We then display the attention heatmap aggregated and normalized across the tokens (it would be difficult to visualize a 242*242 heatmap)

```
with nn_model.trace(example_tokens):  
    for layer_idx in range(cfg.n_layers):  
        pattern = nn_model.blocks[layer_idx].attn.hook_pattern.output.save()  
        attention_patterns.append(pattern)
```

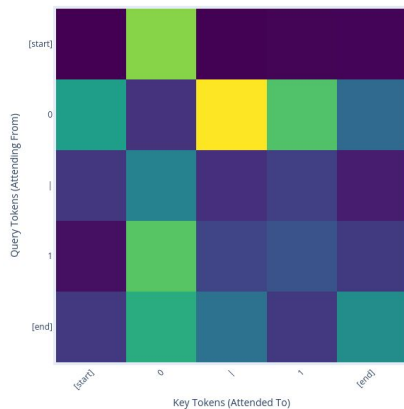
Layer 0, Head 0 Attention Pattern



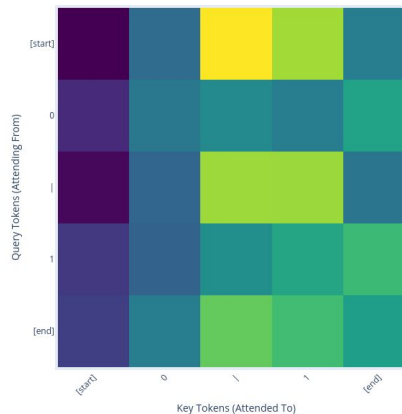
Layer 1, Head 0 Attention Pattern



Layer 0, Head 1 Attention Pattern



Layer 1, Head 1 Attention Pattern



- Layer 0 attention heads has a high attention bias towards “|” (*separating token*). **L0 seems to handle row separation.**
- L1.H0 is mostly handling structure of the polygon (*it only attends to special tokens*).
- L1.H1 seems to start attending to 1 [**maybe a 1-counter circuit?**]

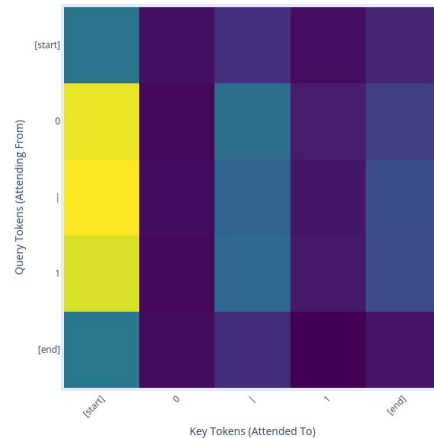
Layer 0 AH

Layer 1 AH

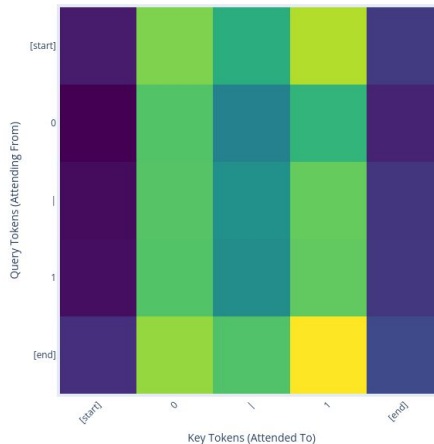
Layer 2 - 0 AND 1 being attended??? ✓

- FINALLY, the L2.H1 starts to attend to the *actual* tokens.
- Hints at L2.H1 being important in the circuit we are trying to find (**hypothesis : there is some “counter” circuit that’s counting 1’s or 0’s**)
- Let’s look at L2.H1’s confidence by sampling some data from the dataset, doing a forward pass for each, and back-computing its contribution from each L2.H1 attention head. We cluster them by cosine similarity and color the frequency distribution.

Layer 2, Head 0 Attention Pattern

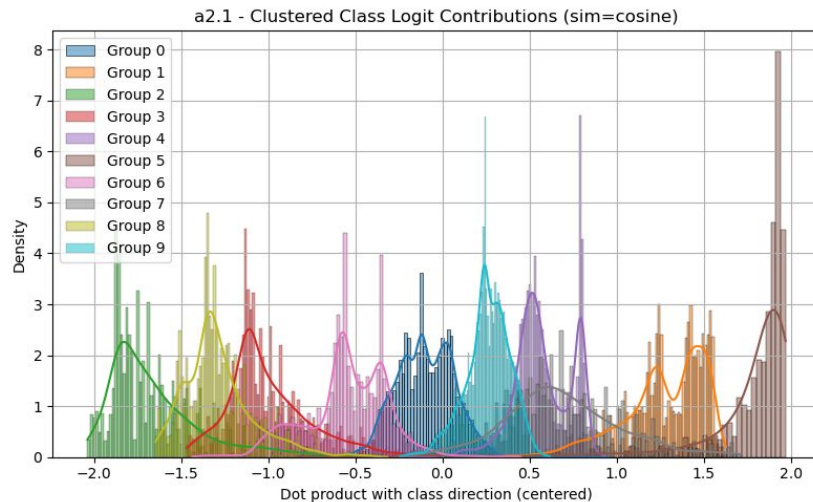


Layer 2, Head 1 Attention Pattern



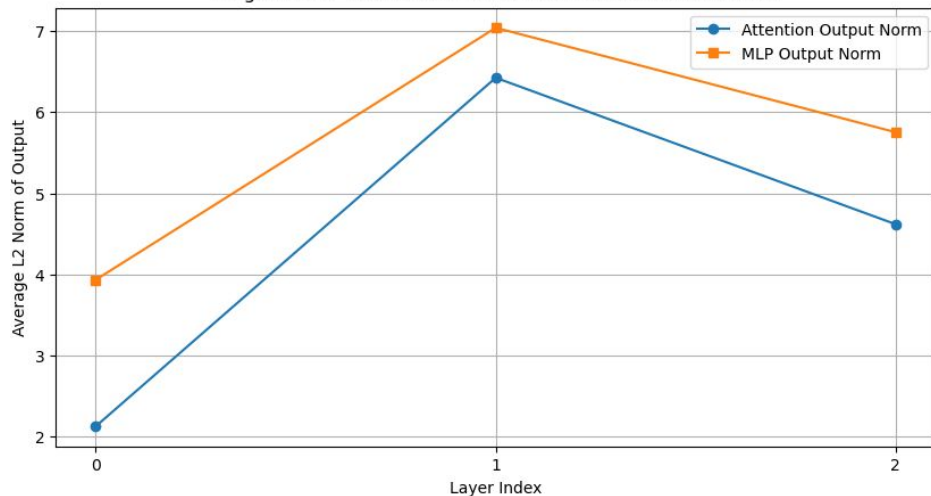
L2.H1 and what even??

- The multi-modal distribution implies that the component is able to move things in some direction of our embedding space.
- The groups here, seem to disjoint unions of separated consecutive classes (similar to the area task : few slides later)



Group 0: [34, 39, 40, 41, 47, 48]
Group 1: [2, 5, 7, 10, 11, 12, 13, 35, 49, 54]
Group 2: [**15, 16, 17, 18, 19**, 27, 30, 31]
Group 3: [22, 23, 28]
Group 4: [33, 42, 43, 44, 45, 46]
Group 5: [**0, 1, 3, 4, 6, 8, 9**, 14, 51, 52, 53]
Group 6: [32]
Group 7: [20, 21, 24, 25, 26]
Group 8: [36, 37, 38, 50]
Group 9: [29]

Magnitude of Attn vs MLP Contributions to Residual Stream



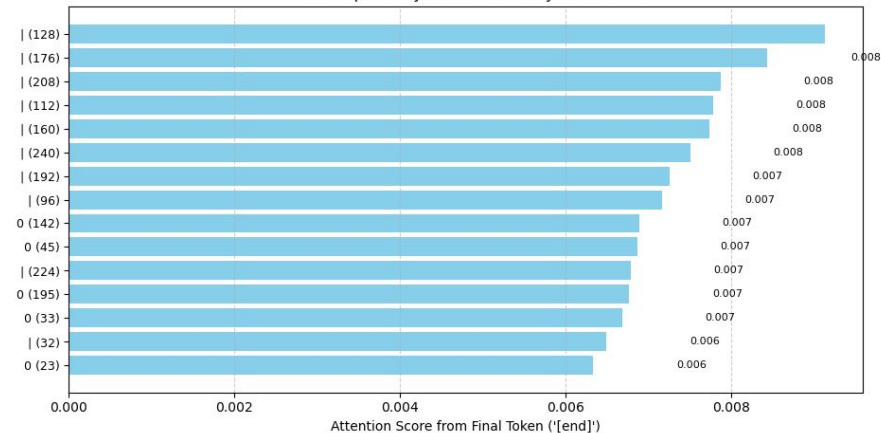
What it measures: The average L2 norm (Euclidean length/magnitude) of the output vectors produced by the *entire* Attention sub-layer and MLP sub-layer in each layer. This average is taken across *all token positions* in the sequence.

Question Answered: "On average across the whole sequence, how 'large' or 'strong' was the update vector produced by the Attention mechanism in this layer? How 'large' was the update from the MLP mechanism?"

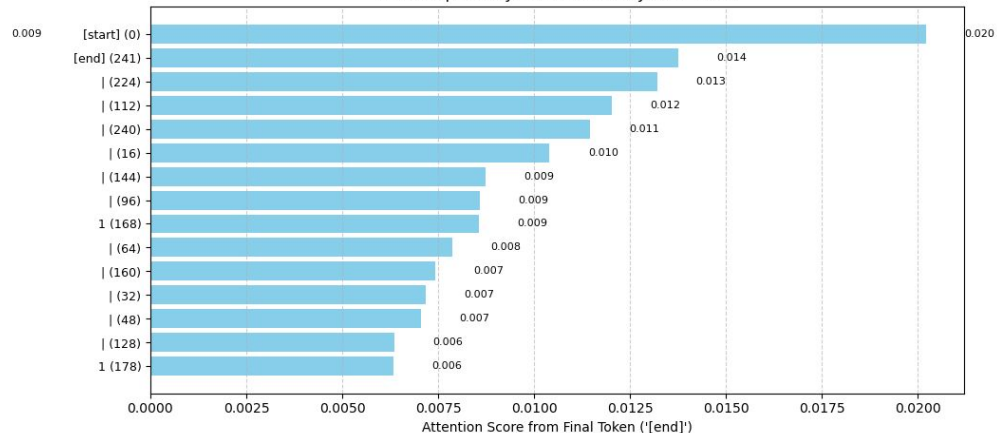
- The MLPs seem to make larger magnitude updates to the residual stream than the Attention mechanisms at every layer.
- Both components exhibit peak activity in the middle layer (Layer 1), suggesting this is where the most significant processing occurs.

```
with nn_model.trace(example_tokens) as trace:
    embed_proxy = nn_model.hook_pos_embed.output.save()
    resid_stream_states_proxies.append(embed_proxy)
    for l in range(model.cfg.n_layers):
        mlp_out_proxy = nn_model.blocks[l].mlp.output.save()
        mlp_contributions_proxies.append(mlp_out_proxy)
```

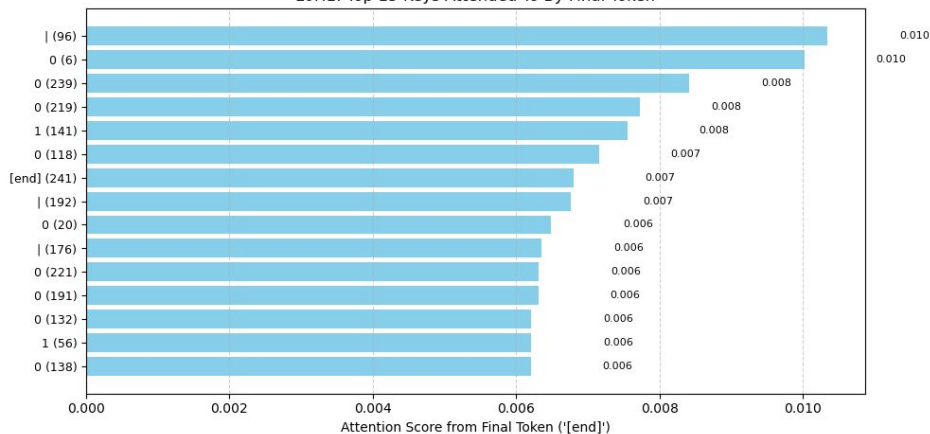
L0H0: Top 15 Keys Attended To By Final Token



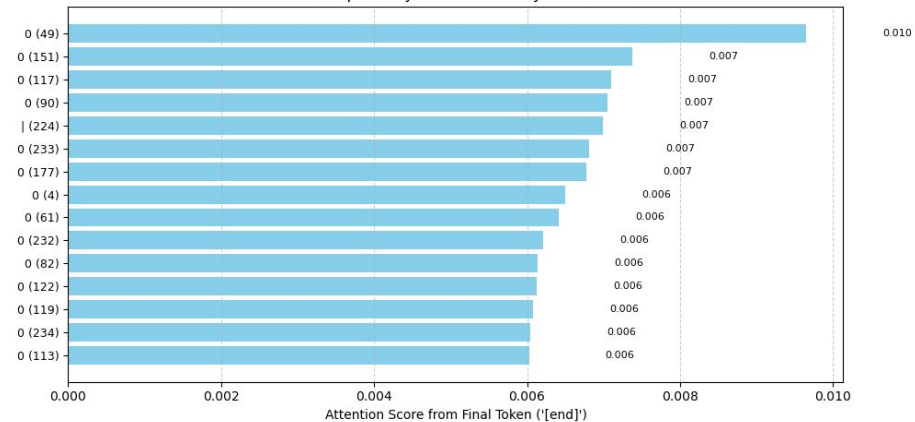
L1H0: Top 15 Keys Attended To By Final Token



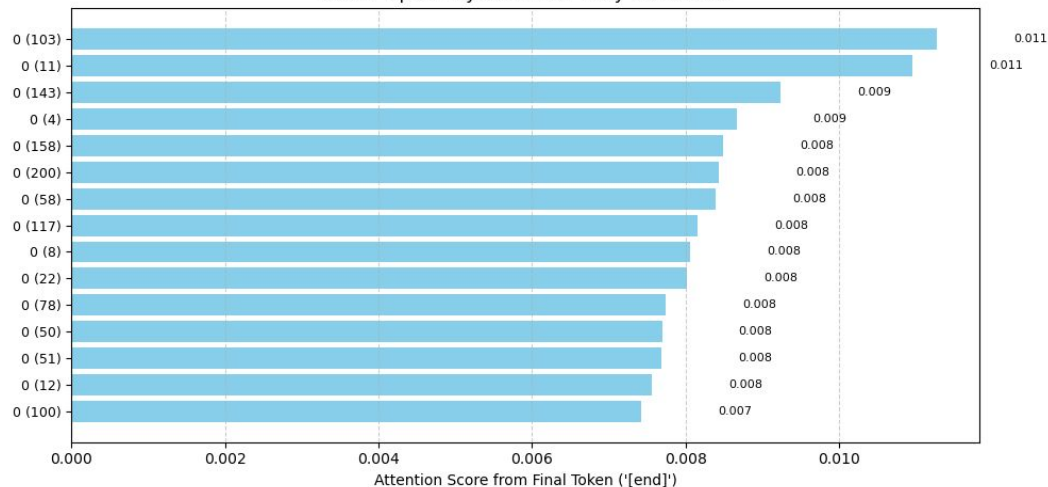
L0H1: Top 15 Keys Attended To By Final Token



L1H1: Top 15 Keys Attended To By Final Token



L2H1: Top 15 Keys Attended To By Final Token



L0H0: Attended to 0 '1' tokens within the top 15.

L0H1: Attended to 2 '1' tokens within the top 15.

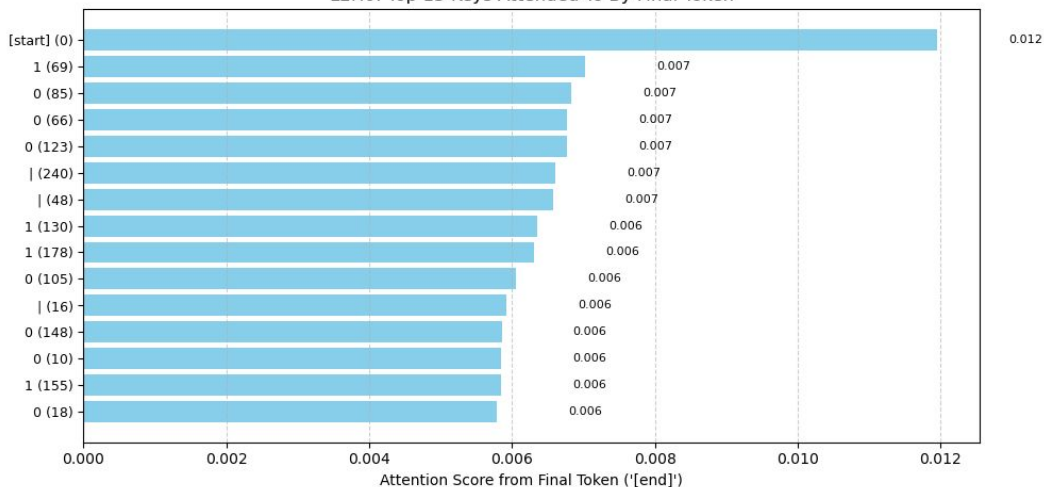
L1H0: Attended to 2 '1' tokens within the top 15.

L1H1: Attended to 0 '1' tokens within the top 15.

L2H0: Attended to 4 '1' tokens within the top 15.

L2H1: Attended to 0 '1' tokens within the top 15.

L2H0: Top 15 Keys Attended To By Final Token

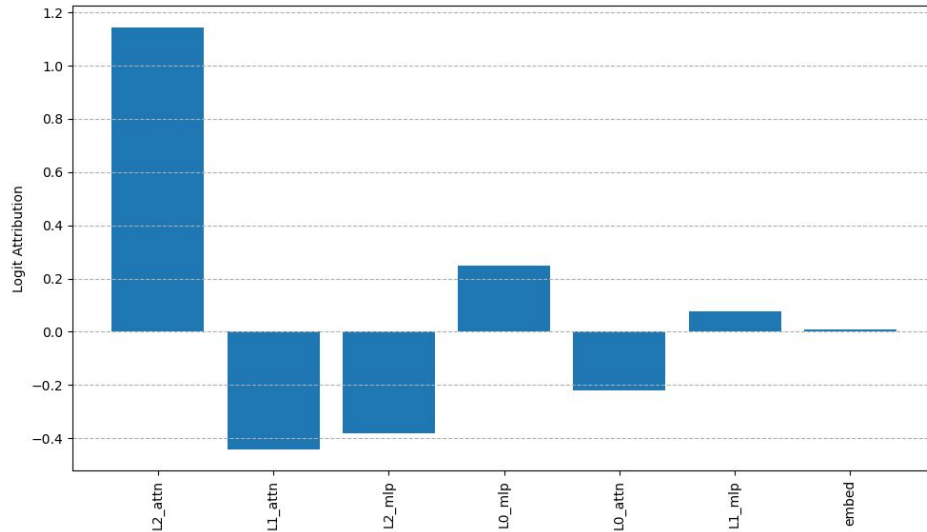


0 has the most attention 🤔

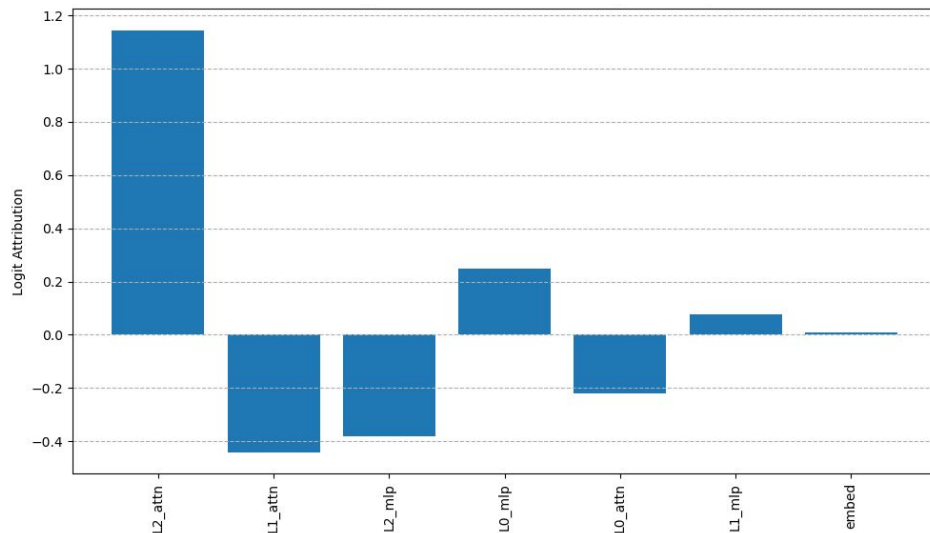
Surprisingly, even though 1 is more sparse, the model seems to be attending only to 0's

What it measures: The contribution of each component's output vector (specifically at the *final token position*) projected onto the *direction* in the residual stream corresponding to the *correct answer logit* (Perimeter 35 in this case).

Question Answered: "How much did this component's output at the final token push the model's state *towards* (positive value) or *away from* (negative value) predicting the correct answer 'Perimeter 35'?"



- **Distributed Counting:**
Multiple heads across layers might share the counting load.
- **Sequential Info Passing:**
'1's could be passed token-by-token, not all to the final token.
- **MLP Role?:**
Uncommon, but MLPs might help tag or increment '1's locally.



L2_attn:

Strong positive ($\sim +1.15$) — **main driver** of correct prediction

L0_mlp:

Moderate positive ($\sim +0.25$)

L1_mlp:

Small positive ($\sim +0.08$)

Embedding + Positional:

Near zero — initial token state was neutral

L2_attn likely aggregates critical info (e.g., number of '1' tokens)

It could:

- Directly gather & encode the count
- Amplify info already present in residual stream

Which **heads in Layer 2** contribute most?

- Use head-level DLA

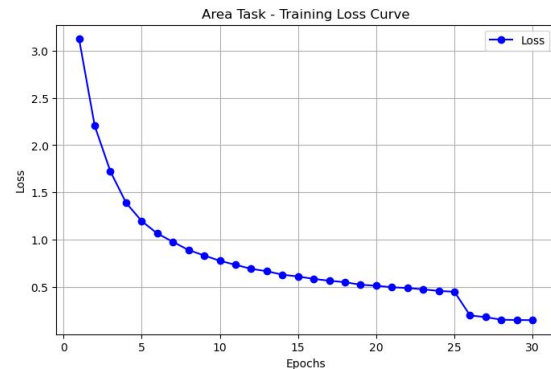
What's in the **residual stream before L2_attn**?

- Analyze incoming info to L2 Attention

Area Transformer

Experiments and Conclusions - Area task

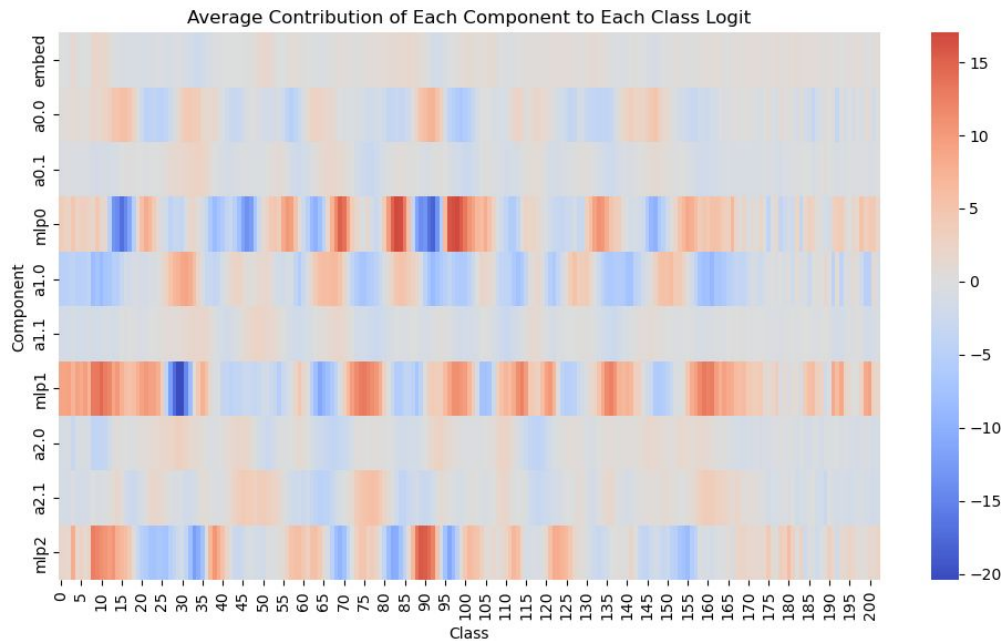
- We are currently at ~**95% accuracy** for the area task using the same toy transformer.
- Good? An actually non-trivial task for which we could train our transformer to start performing really well.
- Hypotheses?



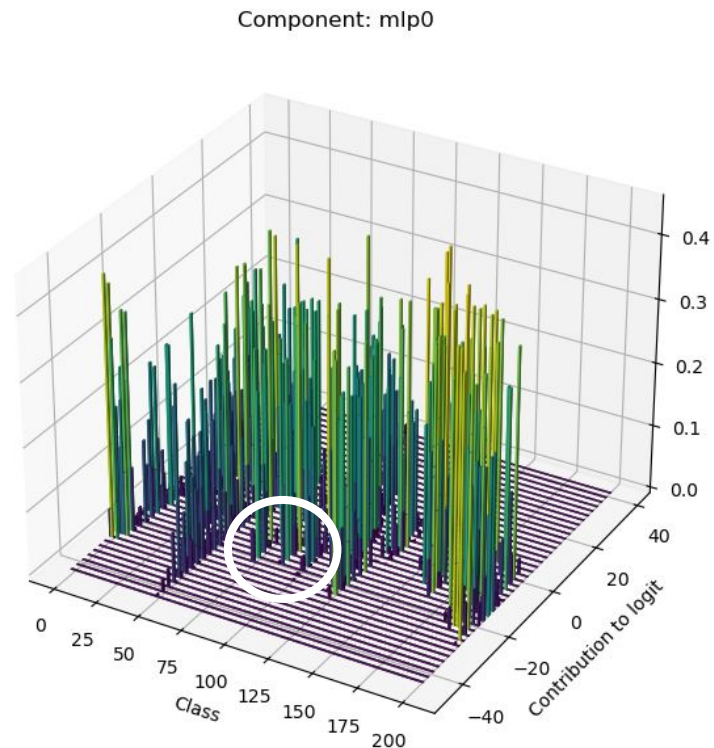
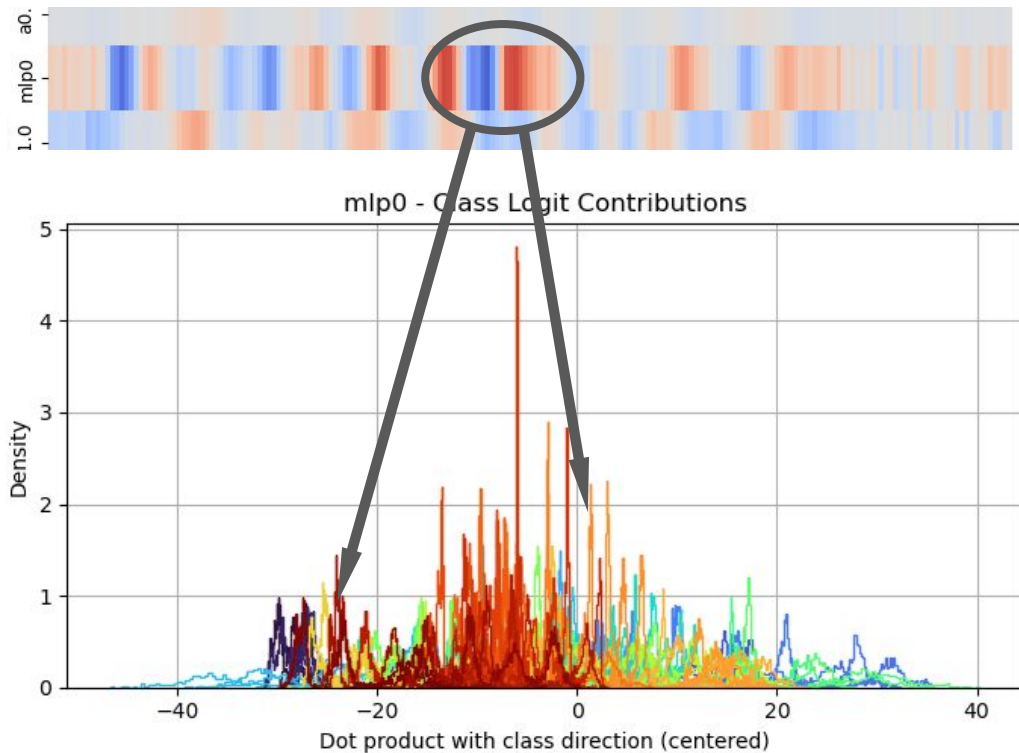
Accuracy: 0.9433

Component Attribution - Area

- Thus, for each model component and for each class (area value in the sampled data); we see how much the component *pushes* it in the direction.
- Observations:
 - MLPs doing a LOT of the heavy lifting (model not using attention much?)
 - Alternating patches of *lighting up* for every component over classes.

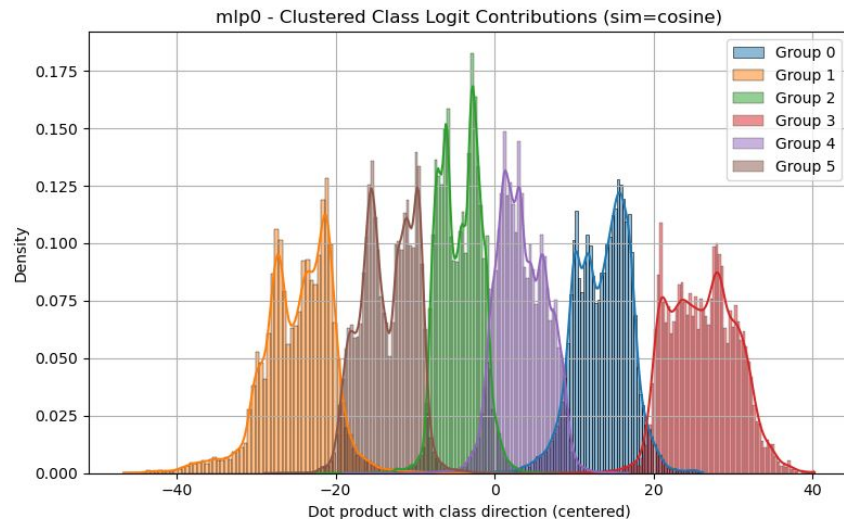
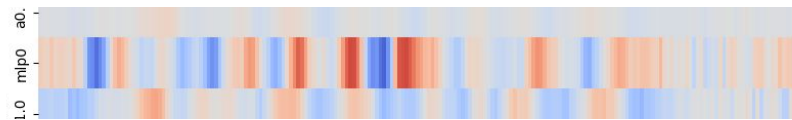


Per class movement from each component - mlp0



Polysemanticity??? - mlp0

- It feels like there are components taking *ownership* of certain area classes.
- We take the logit patterns and do cosine similarity clustering on them. Then, the dot product wrt classes is plotted and coloured by similarity.



Group 0: [16, 26, 31, 32, **40, 41, 42, 43, 44, 45, 47, 48, 61, 67, 79, 80, 81, 82, 84, 85, 86, 87, 88, 104, 105, 106, 147, 148, 149, 150, 151, 152, 153, 178]**

Group 1: [7, 12, 21, **70, 71, 72, 73, 74, 75, 76, 90, 91, 93, 94, 95, 96,** 101, **109, 110, 111, 115, 120, 121, 123, 134, 135, 136, 137, 138, 140, 141, 143, 157, 158, 160, 162, 163, 164, 165, 166,** 193]

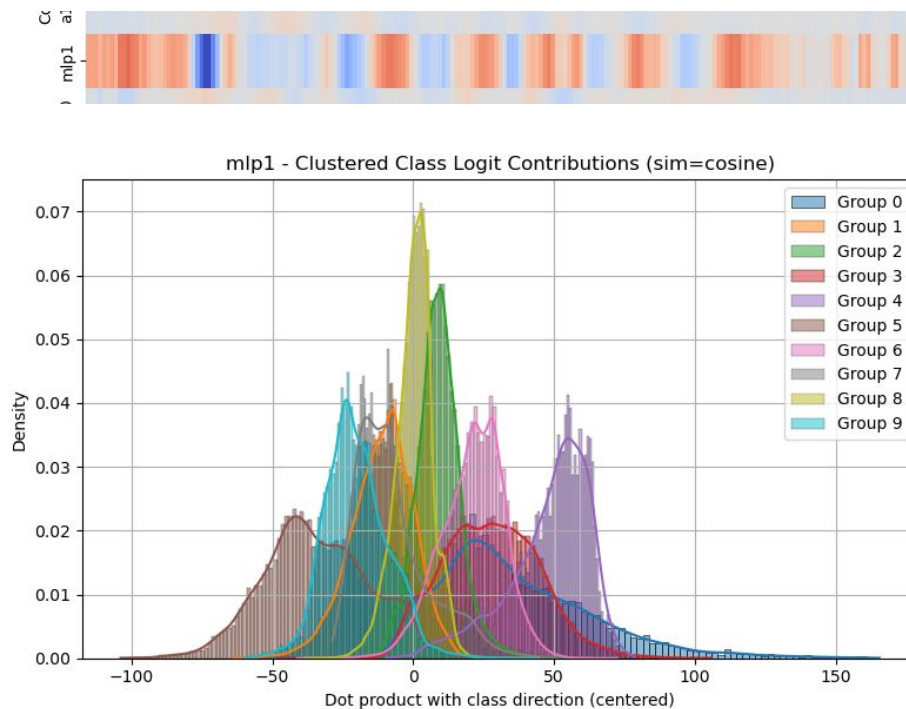
Group 2: [13, 14, 20, 22, 23, **33, 34, 35, 36,** 53, 59, 69, 77, 102, 108, 119, 124, 133, **139, 142, 144, 145, 146,** 155, 156, **167, 168, 169,** 173, 177, 185, 186, 188, **192, 194, 195, 197, 198, 201, 202]**

Group 3: [**27, 28, 29, 30, 62, 63, 64, 65, 66,** 83]

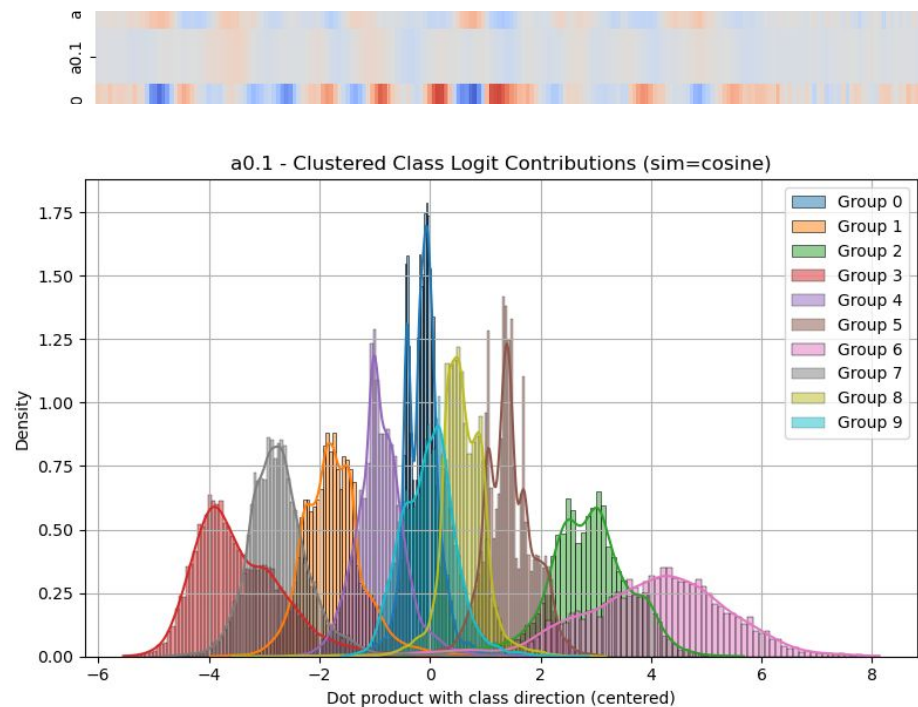
Group 4: [**0, 1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 92, 97, 98, 99, 100,** 112, 113, 114, 122, 159, 161, 191, 199, 200]

Group 5: [**15, 17, 18, 19, 24, 25, 37, 38, 39, 46, 49, 50, 51, 52, 54, 55, 56, 57, 58, 60, 68, 78, 89, 103, 107, 116, 117, 118, 125, 126, 127, 128, 129, 130, 131, 132,** 154, **170, 171, 172, 174, 175, 176, 179, 180, 181, 182, 183, 184, 187, 189, 190,** 196]

Others? -> Not as clear what they do (class confidence?) 🤔



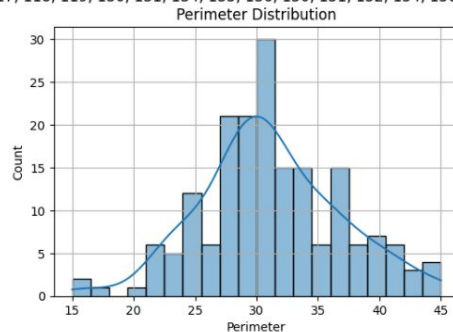
High activation bias, low inductive bias??



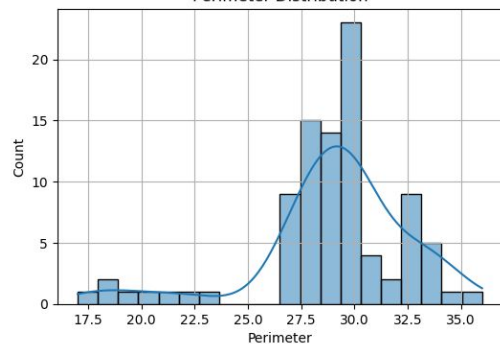
Low activation bias, High inductive bias??

New Hypothesis

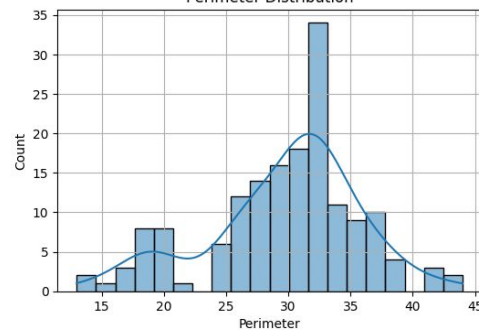
Component=3, Cluster=0
Classes=[12, 22, 23, 24, 36, 41, 45, 55, 57, 59, 70, 71, 72, 73, 78, 79, 93, 99, 114, 115, 116, 117, 118, 119, 130, 131, 134, 135, 136, 150, 151, 152, 154, 156, 157, 161, 163, 164, 170, 171, 173, 174, 175, 176, 177, 178, 183, 184, 187, 197, 198] (182 examples)



Component=3, Cluster=1
Classes=[27, 28, 29, 30, 82, 83, 84, 85] (91 examples)



Component=3, Cluster=3
Classes=[14, 15, 16, 17, 18, 19, 20, 26, 31, 32, 62, 65, 66, 81, 86, 95, 96, 105, 106, 107, 138, 139, 140, 141, 142, 143, 144, 145] (162 examples)



Hypothesis - Maybe layers encode perimeter info?

Probing Task

- Due to the multiple consecutive clusters in each component, we thought they encode some perimeter information
- And then the mlp layers map them to area clusters
- Rounded peri values to nearest 5 multiple
- Potentially be encoding perimeter information

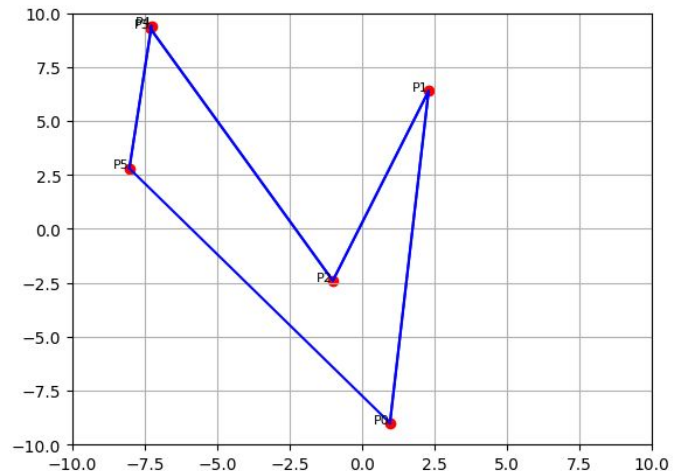
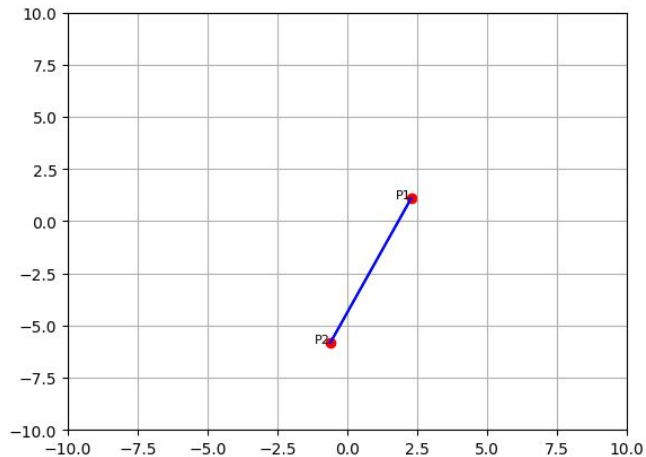
Perimeter	Classification		Accuracy: 0.907		
	precision	recall	f1-score	support	
10	0.00	0.00	0.00	0	
15	1.00	0.80	0.89	10	
20	0.94	0.85	0.89	60	
25	0.90	0.89	0.90	222	
30	0.92	0.90	0.91	370	
35	0.89	0.96	0.92	255	
40	0.94	0.86	0.90	77	
45	0.60	1.00	0.75	6	
accuracy			0.91	1000	
macro avg	0.78	0.78	0.77	1000	
weighted avg	0.91	0.91	0.91	1000	

(Short) vertices task

Alternate Dataset Generation

With vertices instead of grid points

1. Two points forming a line, and the length of the line segment (as a prelim task)
2. Variable number of vertex points forming a variable sided polygon and the perimeter of the polygon



Initial Hypothesis

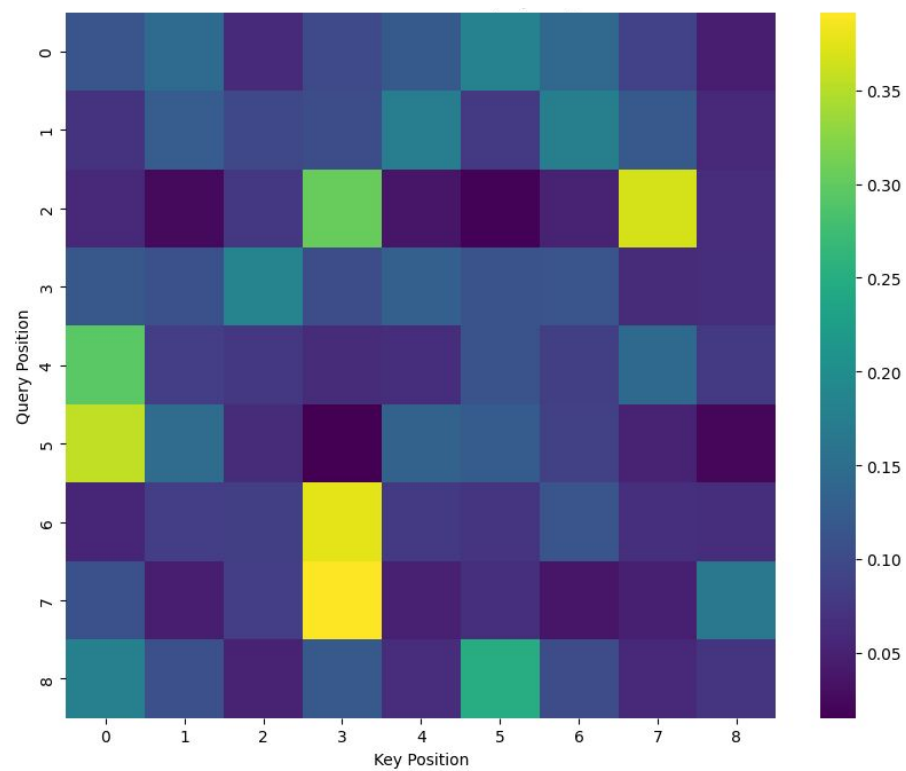
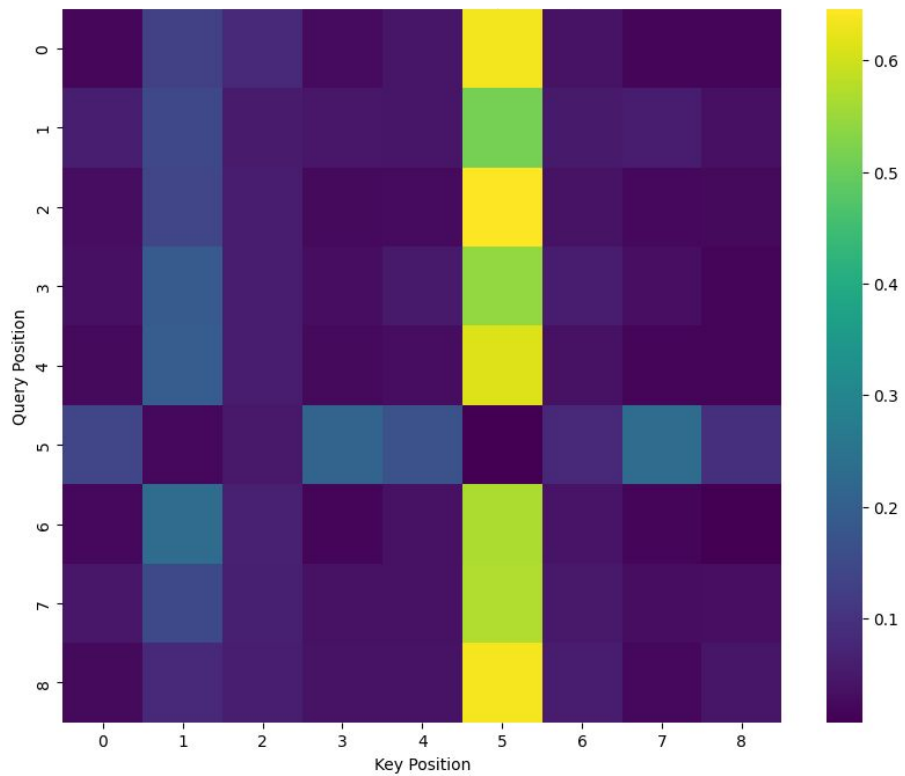
Position	Token
0	[start]
1	'1'
2	' '
3	'9'
4	' '
5	'8'
6	' '
7	'0'
8	[end]

We found the attention layers in the transformer, pay a lot of attention of the point of change between coordinates

We found that a lot of attention is funneled into token 5, the first token of the second point.

A pattern like “most rows → column 5” suggests that various tokens—especially [start], '|'—are funneling attention onto '8'. Possibly, in your model’s learned logic, the '8' token is relevant for gating the next step in the distance computation.

The first layer is diffused and could be capturing local token context



Thank you!

More References

- [Neel Nanda's 300+ open problem ideas on Mech Interp](#)
- [The clock and pizza paper](#)
- [Representation learning paper from MIT](#)
- [GPT-2-small study on circuit for "greater-than"](#)
- [Neel Nanda et. al's reverse engineering modular addition paper](#),
a [walkthrough blog](#) and a [video walkthrough](#) for the same.
- [Neel Nanda et. al's reverse engineering binary group operations](#)

Old slides ahead (JIC)

Related Work

NeurIPS '22

Towards Understanding Grokking: An Effective Theory of Representation Learning

Ziming Liu, Ouail Kitouni, Niklas Nolte, Eric J. Michaud, Max Tegmark, Mike Williams
Department of Physics, Institute for AI and Fundamental Interactions, MIT
{zmliu, kitouni, nnolte, ericjm, tegmark, mwill}@mit.edu

NeurIPS '23

How does GPT-2 compute greater-than?: Interpreting mathematical abilities in a pre-trained language model

Michael Hanna*
ILLC
University of Amsterdam
m.w.hanna@uva.nl

Ollie Liu*
University of
Southern California
zliu2898@usc.edu

Alexandre Variengien†
Redwood Research
alexandre.variengien@gmail.com

ICLR '23 (spotlight)

PROGRESS MEASURES FOR GROKING VIA MECHANISTIC INTERPRETABILITY

Neel Nanda*, † Lawrence Chan‡ Tom Lieberum† Jess Smith† Jacob Steinhardt‡

NeurIPS '23

Towards Automated Circuit Discovery for Mechanistic Interpretability

Arthur Conmy* Augustine N. Mavor-Parker* Aengus Lynch* Stefan Heimersheim
Independent UCL UCL University of Cambridge

Adrià Garriga-Alonso*
FAR AI

NeurIPS '23 (oral)

The Clock and the Pizza: Two Stories in Mechanistic Explanation of Neural Networks

Ziqian Zhong*, Ziming Liu*, Max Tegmark, Jacob Andreas
Massachusetts Institute of Technology
{ziqianz, zmliu, tegmark, jda}@mit.edu

ICML '23

A Toy Model of Universality: Reverse Engineering how Networks Learn Group Operations

Bilal Chughtai¹ Lawrence Chan² Neel Nanda¹

Related Work

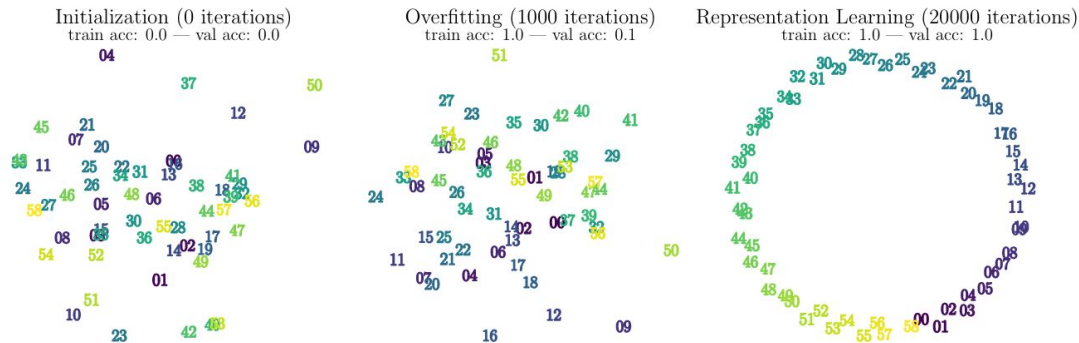


Figure 1: Visualization of the first two principal components of the learned input embeddings at different training stages of a transformer learning modular addition. We observe that generalization coincides with the emergence of structure in the embeddings. See Section 4.2 for the training details.



Figure 2: The train and test accuracy (left) and train and test loss (right) of one-layer transformers on the modular addition task described in Section 3, over 5 random seeds. These models consistently exhibit grokking: they quickly overfit early on in training, but then later learn to generalize.

Background Work on Mech Interp

We have been going through Callum McDougal and Neel Nanda's [Mechanistic Interpretability arena](#). Their notebooks outline various tools and techniques that we can use to study the models that we have trained on certain tasks.

This took us *a. lot. of. background. reading.* The field has barely any resources other than the arena notebooks and there are many techniques that are used.

The ad hoc nature of everything here forces us to see as many examples as we can of folks reverse engineering tasks and use similar tools.

[1.2] Intro to Mechanistic Interpretability: TransformerLens & induction circuits

[1.5.1] Balanced Bracket Classifier

Monthly Algorithmic Problems

This is the homepage for the ARENA Monthly Algorithmic Problems sequence. I designed them in the spirit of [Stephen Casper's challenges](#), but with the more specific aim of working well in the context of the rest of the ARENA material, and helping people put into practice all the things they've learned so far.

The current live problem is Trigrams (Nov 24). You can also browse through all past problems below, to see a brief description and summary of the problem. You can navigate between problems using the left-hand sidebar.

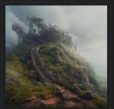
Click to navigate through past monthly problems:



Trigrams (Nov 24)



Caesar Cipher (Jan 24)



Cumulative Sum (Nov 23)



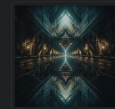
Sorted List (Oct 23)



4-Digit Sum (Sep 23)



Unique Character (Aug 23)



Palindromes (Jul 23)

Background Work on Mech Interp

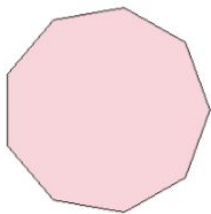
- For example, the interpretation of the balanced bracket classifier task in the arena notebooks made them realize that the toy transformer is learning a “total elevation circuit” (ensuring that the prefix sum of the bracket sequence is never negative and ends at 0).
- To realize this they make a lot of ad hoc “guesses” and then use ablations like logit attribution, finding head influence and responsible heads etcetc.
- Point being, we now have the models at hand, we need to start scratching our heads and “guessing” model behaviour and applying the analyses that these folks do :)

Testing Pretrained Vision Transformers and VLMs

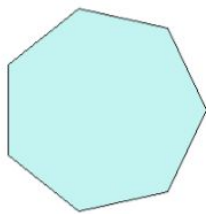
1. We ran inference on pre trained vision transformers, Resnet, vit, swin
2. Asked it to count the number of vertices of the polygon, given the image (regular polygons were used) and it give very poor accuracy ~ 14%.
3. It didn't even predict a lot of classes. the sides of polygons varied from 3 to 12 and it didn't predict any polygon with sides 3,5,9,10,12.
4. But after fine tuning with 1 epoch, roughly 100 images per polygon, it gave almost 100% accuracy.

Model	Accuracy
ResNet50	14.72
ViT-Base	10.24
SWIN	0.14%

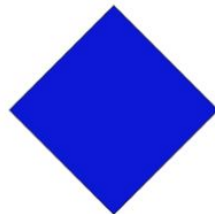
True: 9.0
FT: 9
Base: 7



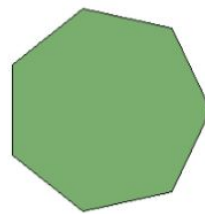
True: 7.0
FT: 7
Base: 4



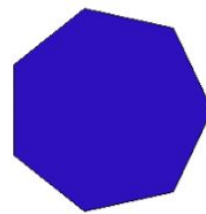
True: 4.0
FT: 4
Base: 8



True: 7.0
FT: 7
Base: 4



True: 7.0
FT: 7
Base: 7



Confusion Matrix:

```
[[325  0  0  0 574 62  0  0  0 39]
 [ 80  0  0  3  0 803  0 15  0 99]
 [ 98  0  0  0  0 834  0  8  0 60]
 [  4  0  0  0  0  76  0  0  0 920]
 [ 27  0  0  0  0 547  0  0  0 426]
 [  7  0  0  0  0 831  0  0  0 162]
 [  1  0  0  0  0 676  0  4  0 319]
[146  0  0  0  3 491  0 90  0 270]
 [  2  0  0  0  0 457  0  2  0 539]
[112  0  0  0  1 543  0 118  0 226]]
```

Fine-Tuned Confusion Matrix:

```
[[208  0  0  0  0  0  0  0  0  0]
 [  0 202  0  0  0  0  0  0  0  0]
 [  0  0 192  0  0  0  0  0  0  0]
 [  0  0  0 201  0  0  0  0  0  0]
 [  0  0  0  0 209  0  0  0  0  0]
 [  0  0  0  0  0 186  0  0  0  0]
 [  0  0  0  0  0  0 211  0  0  0]
 [  0  0  0  0  0  0  0 204  0  0]
 [  0  0  0  0  0  0  0  0 212  0]
 [  0  0  0  0  0  0  0  0  0 175]]
```

VLM (CLIP)

1. Experiments run at 3-8 sided regular, Zero-shot inference was done
2. So we gave CLIP some candidate prompts at first, like 3 sided polygon, 4 sided polygon and so on.
3. It gave really bad performance. Like other pretrained models, it didn't predict some classes entirely too.
4. We introduced some candidate prompts for these classes like 5 sided polygon, pentagon and then again tried. Nothing changed. Took it a step further and added such candidate prompts for all polygons like 3 sided polygon, triangle, 4 sided, square and so on.
5. This changed the outputs of the model. It gave more correct predictions but still very random.

